

The HexaPedal

Section HM

Phase IV - 2025-11-24

Mohamad Addasi (40278616)

Fouad Elia (40273045)

Junior Boni (40287501)

Tristan Girardi (40272203)

Youssef Yassa (40265083)

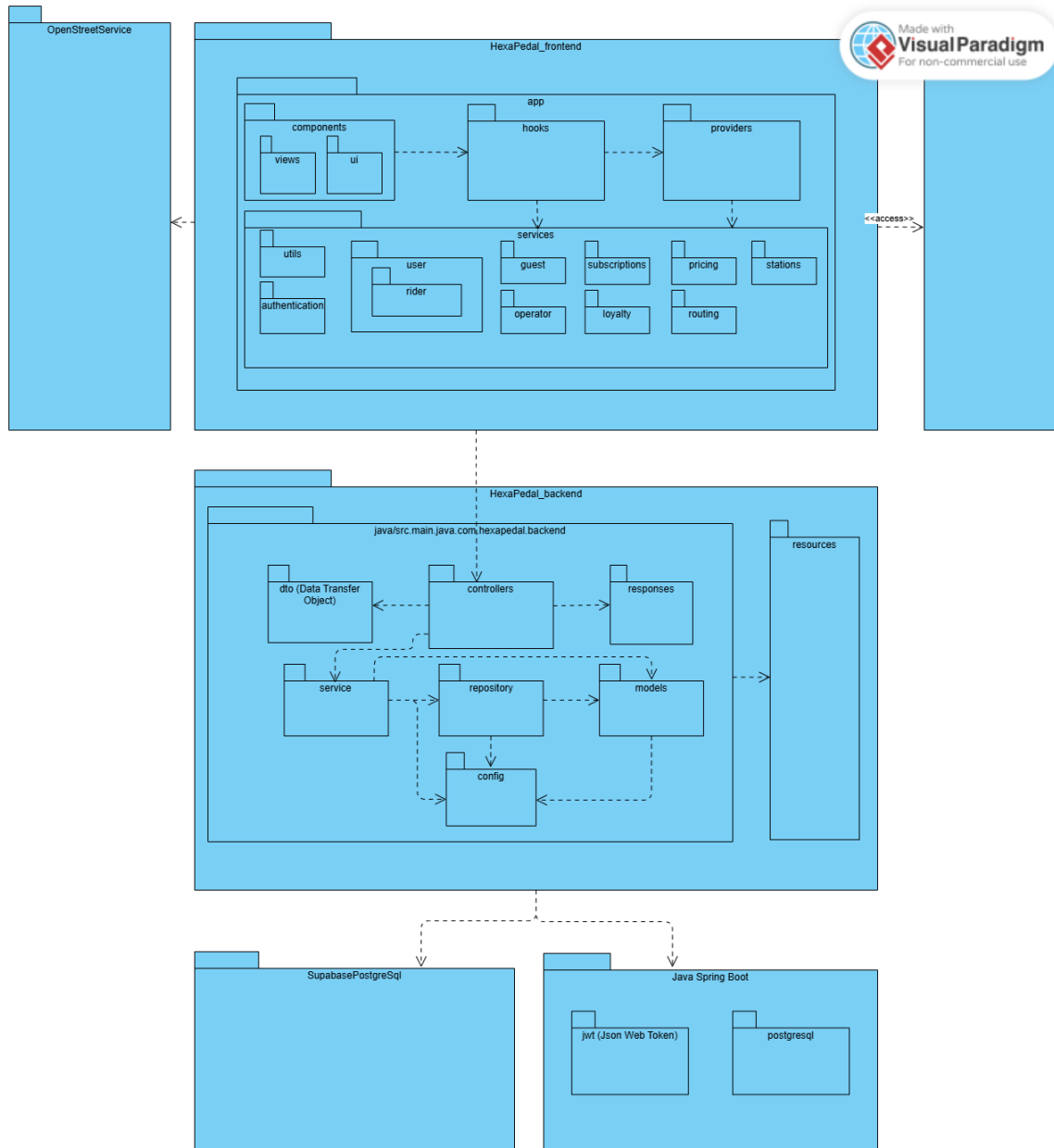
Youssef Youssef (40285384)

Table of Contents

Table of Contents	1
System Architecture (Reviewed)	2
Overview	3
Layers Composition	3
Presentation Layer	4
Application Layer	4
Domain Layer	4
Low-Level Infrastructure Layer	5
Technical Service Layer (Crosscutting layer)	5
Architecture Changes	6
Presentation Layer Service Partitioning	6
Revision of the Low-Level Infrastructure Layer and Technical Layer	6
Domain Model (Reviewed)	7
Summary Critical Changes	8
Design Pattern (Reviews)	8
Events - Factory Method	9
Use Case Model	9
Use Case Diagrams	10
User Switching Views (Operator and Rider)	10
Returning bike at low occupancy station	10
Use Cases Textual Descriptions	11
User Switching Views (Operator and Rider)	12
Returning bike at low occupancy station	13
Sequence Diagrams	15
User Switching Views (Operator and Rider)	16
Returning bike at low occupancy station	16
Contribution Self-Declaration	17

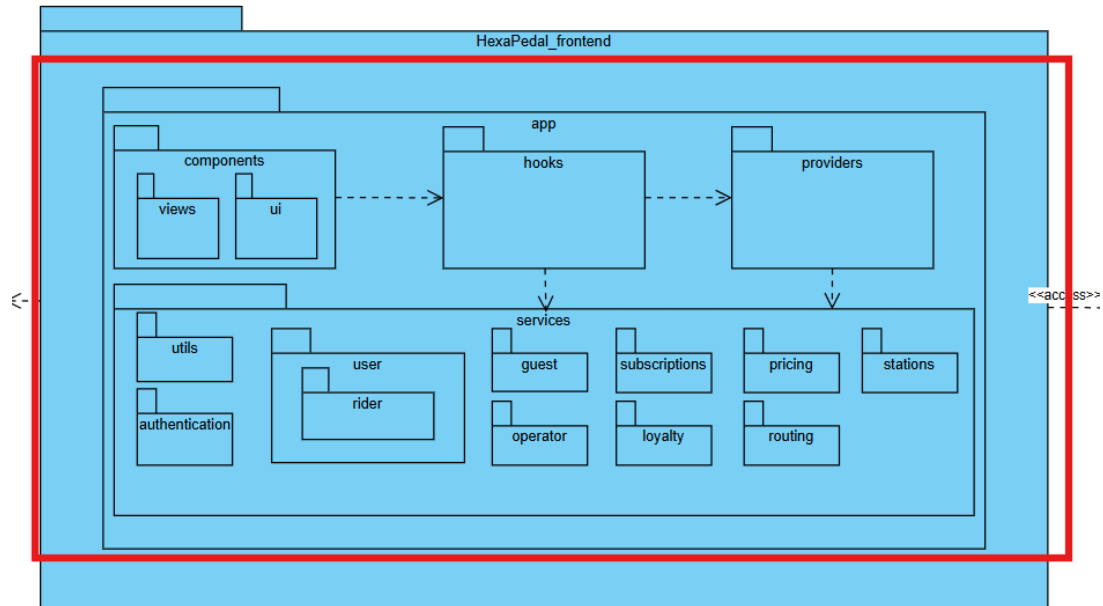
System Architecture (Reviewed)

Overview

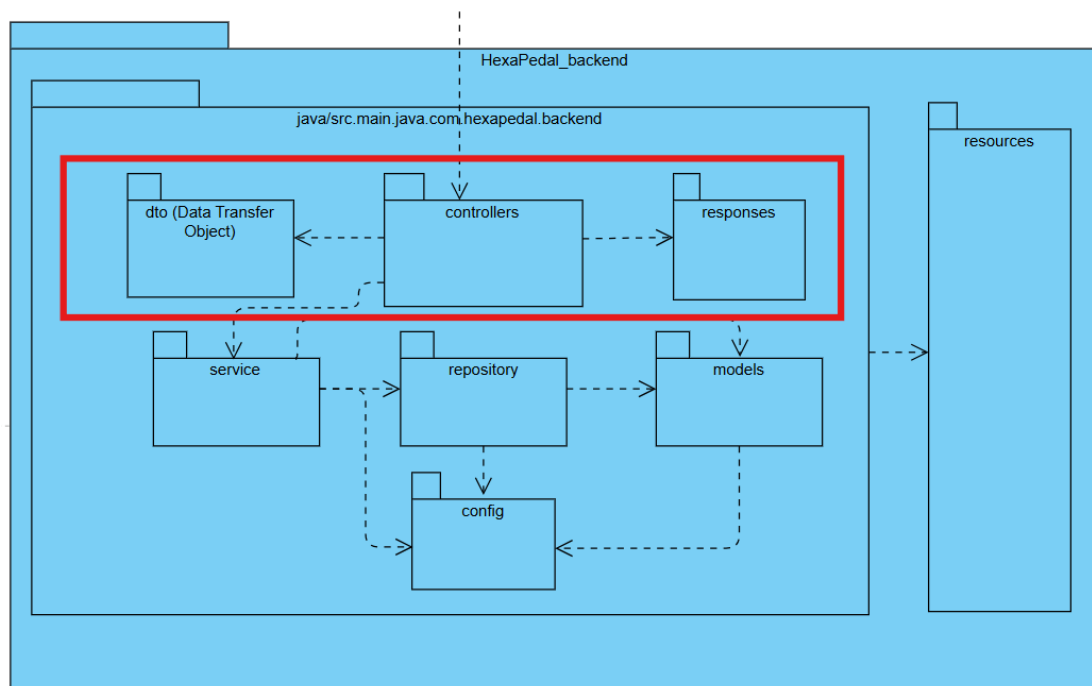


Layers Composition

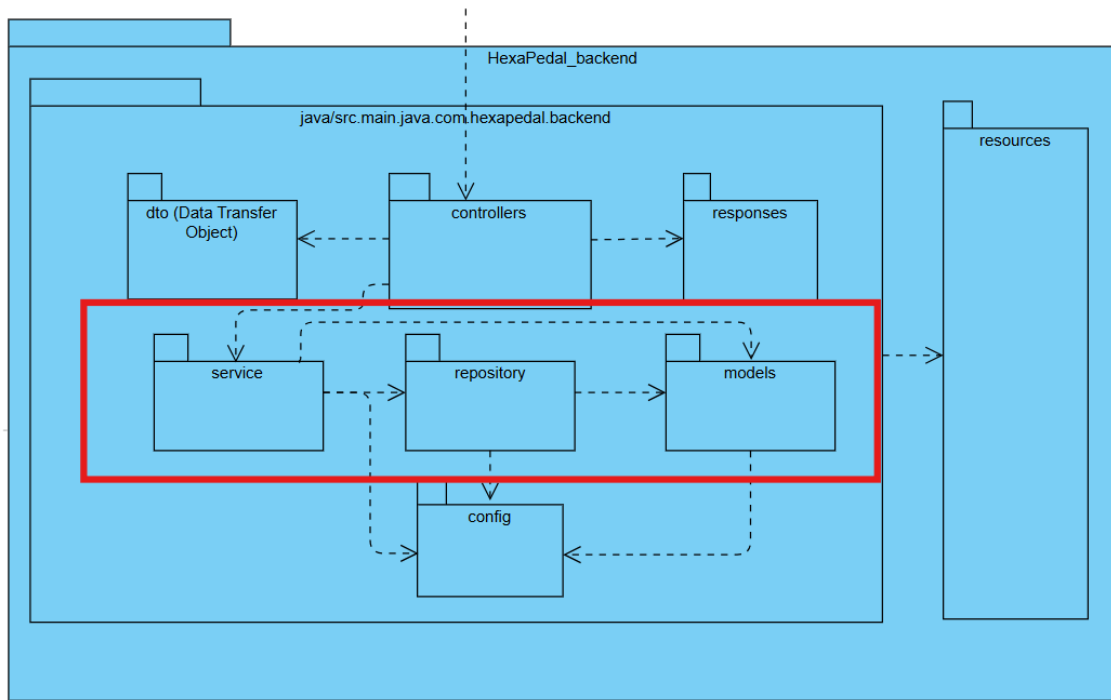
Presentation Layer



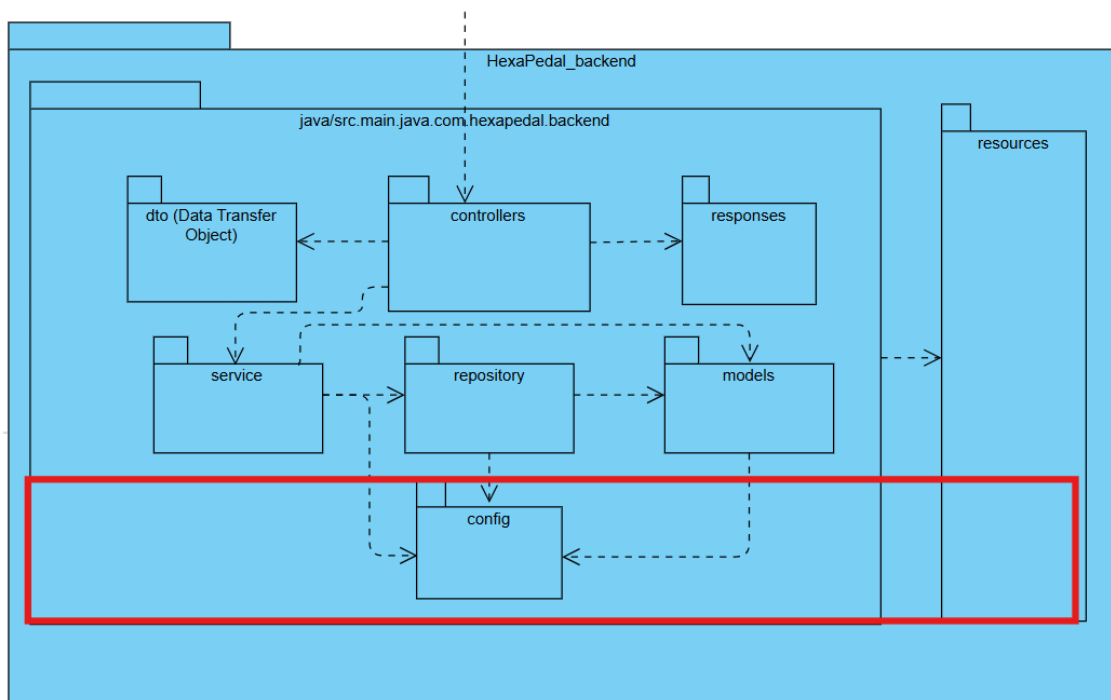
Application Layer



Domain Layer



Low-Level Infrastructure Layer



Technical Service Layer (Crosscutting layer)



Architecture Changes

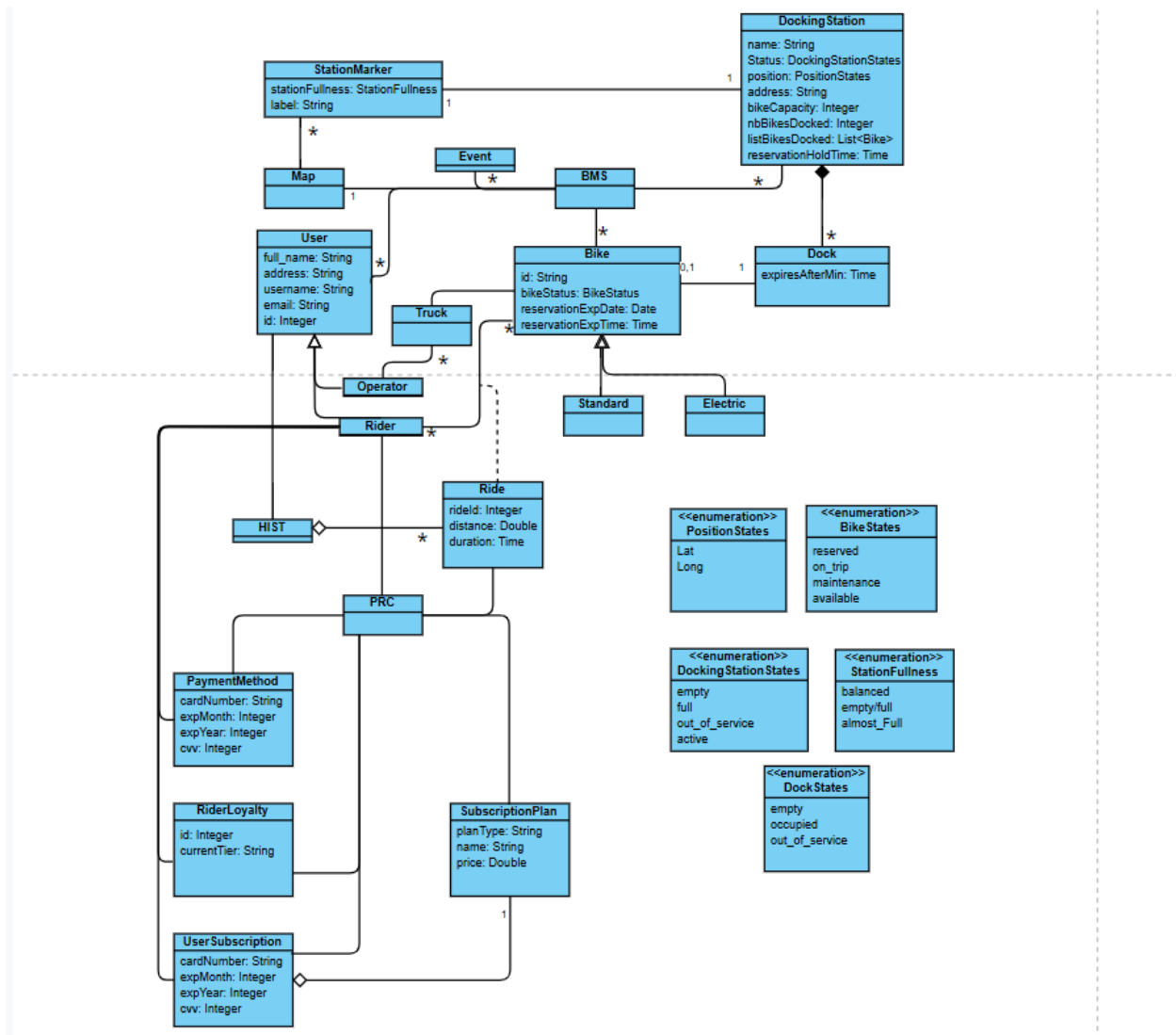
Presentation Layer Service Partitioning

Due to the additional requirements added and to the limit of our foresight, the presentation layer's services have been partitioned into cohesive domain related functionalities in an effort to increase maintainability. This gave birth to sub packages such as *guests*, *subscriptions*, *operators*, *pricing*, *stations*, *loyalty*, and *routing*.

Revision of the Low-Level Infrastructure Layer and Technical Layer

Due to the lack of experience with the Java Spring Boot framework, we included the resources in the source package, which turned out to be the proper practice with the framework. *Resources* is a helper package for Spring Boot; hence, it got removed from the low-level infrastructure layer and added to the technical service layer.

Domain Model (Reviewed)



Summary Critical Changes

- Abstraction of the PRC Service into a single conceptual class accessed by the Rider. This conceptual class contains the logic to handle payment methods, rider's loyalty, and the users' subscription.
- Dock contains 0, or 1 bike
- The *ride* conceptual class is now denoted as an association class between riders and bikes.
- Trucks are dissociated from the stations. They are associated with the bikes they might hold.

Design Pattern (Reviews)

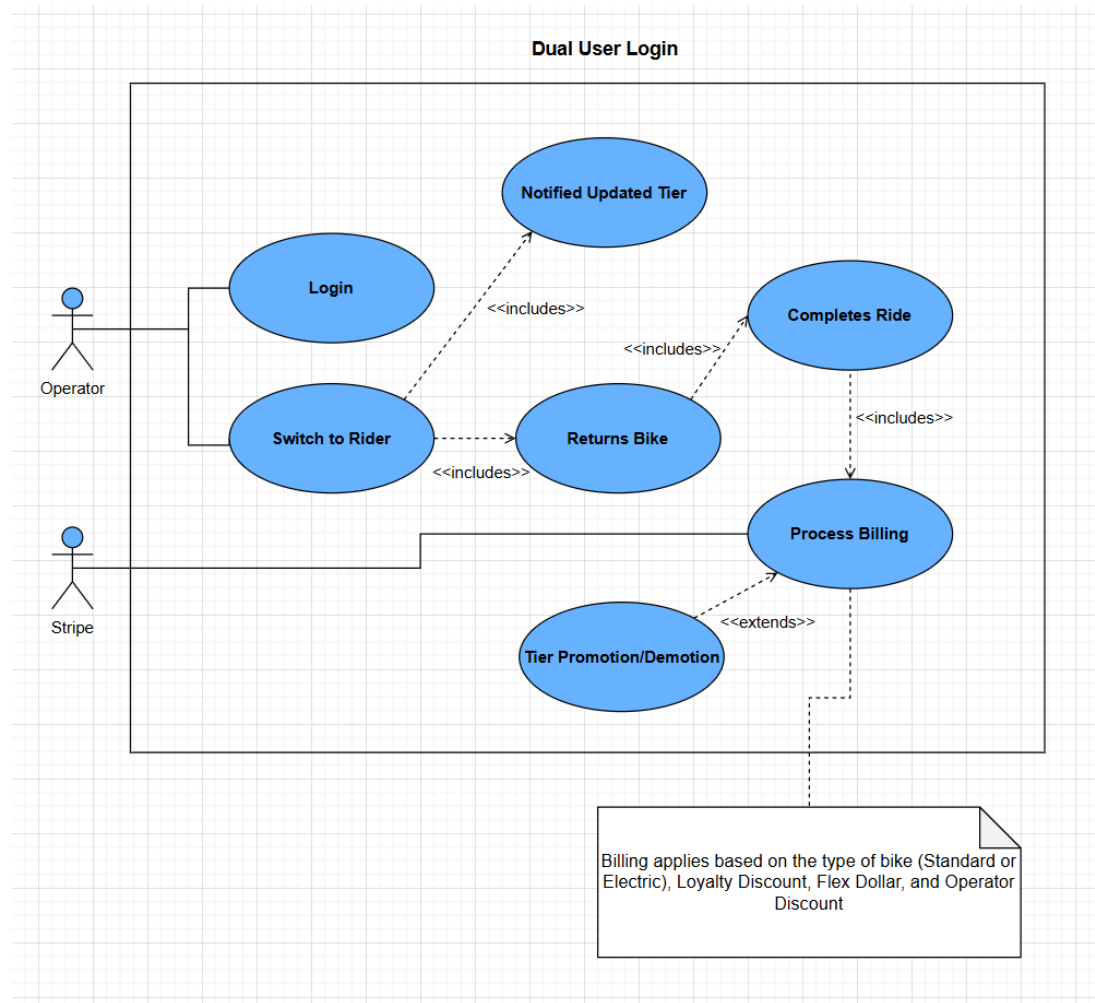
~~Events – Factory Method~~

In [Phase 3](#), we introduced a new design pattern for events logs; however, it was not a pragmatic solution for our use cases, and it was introducing both complexity, which would impede maintenance, because we were introducing new concrete types. Instead, we decided to use the different records that we had of all the different elements and listed them for the operator, and, since Supabase by default can record entry time, we used these features to display them as logs. For instance, thanks to the *rides* and the *users* table, the operators have a list of the trips that happened, as well as the trips info as well as updated state of users' info. Notice, however, that domain wise, grouping the rides and the user under a table did not make much sense. Therefore, we decided to keep it simple.

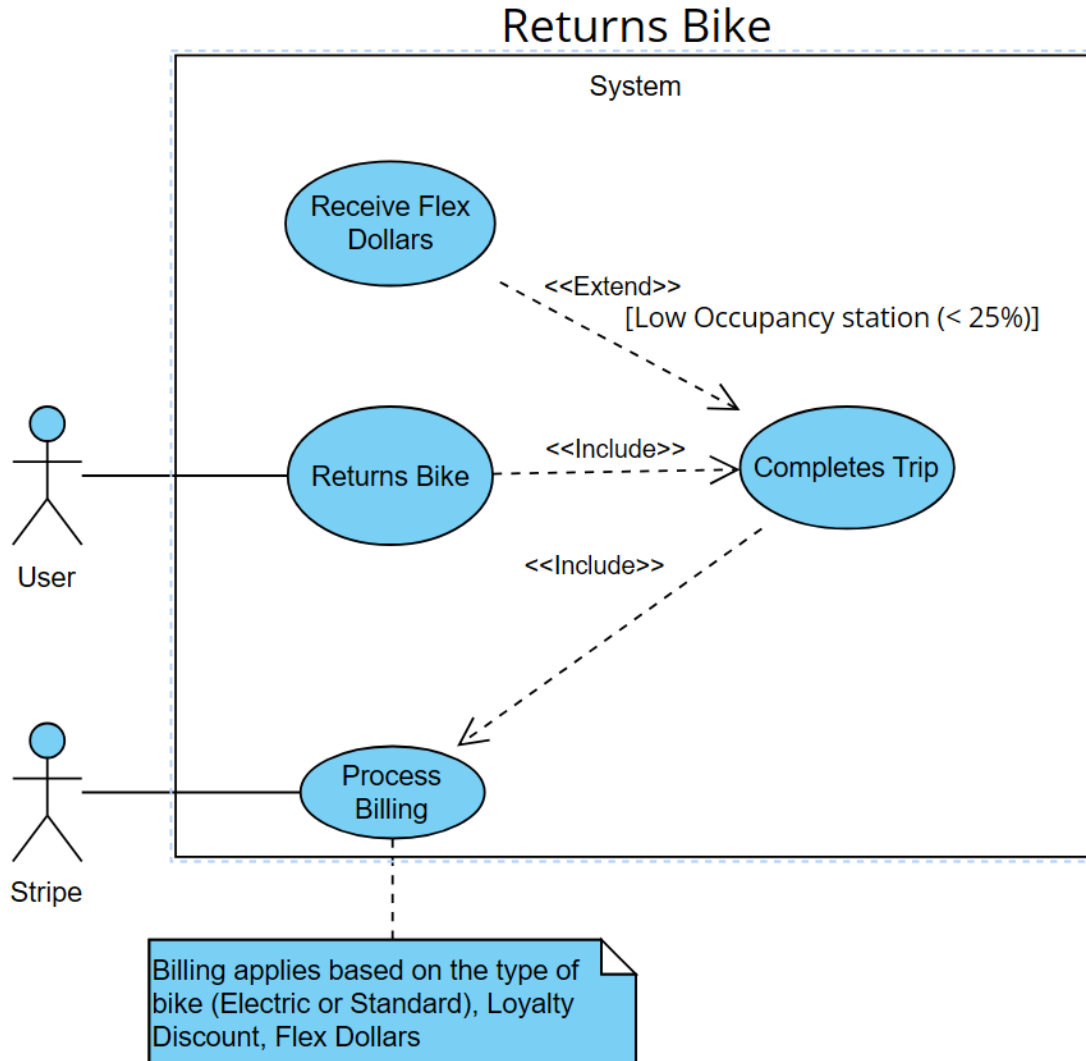
Use Case Model

Use Case Diagrams

User Switching Views (Operator and Rider)



Returning bike at low occupancy station



Use Cases Textual Descriptions

User Switching Views (Operator and Rider)

Use Case Parameters	Details
Name	User Switching Views (Operator and Rider)
Id	1
Actors	Primary Actors: Application User of type Operator
Description	When an operator logs into his account, he will first see the operator's dashboard in which he can perform all the necessary operators' actions. The operator will also have the chance to switch to rider dashboard and behave like a rider so that he could ride bikes if he wanted too
Preconditions	- Operator must be authenticated and logged into the system - Operator must have access to all the Operator actions - Operator is implicitly allowed to act as Rider
Stimulus/Trigger	The mode toggle button ("Rider Mode" button) is clicked in the operator dashboard header
Postconditions	- Dashboard view switched from operator mode to rider mode. - Operator actions become inaccessible until switching back to Operator Mode. - The Operator can access all rider features (reserving bikes, ride bike, etc.)
Basic Flow/Happy Path	1. Operator logs into the system and is redirected to operator dashboard 2. Operator clicks the "Rider Mode" toggle button in the headers 3. Rider dashboard loads with rider-specific features 4. Operator can now perform rider actions 5. Operator can click "Operator Mode" button to switch back to operator dashboard
Alternate Path	None
Exceptions	- If Operator in "Rider Mode" he cannot perform any of the operator task, he must switch back to "Operator Mode" - If Operator in "Operator Mode" he cannot perform any of the rider tasks, he must switch to "Rider Mode"
Minimal Guarantee	- Operator can always return to operator dashboard view - The system preserves correct role permissions based on the current mode

Author	Fouad Elian
Date	November 24 th , 2025

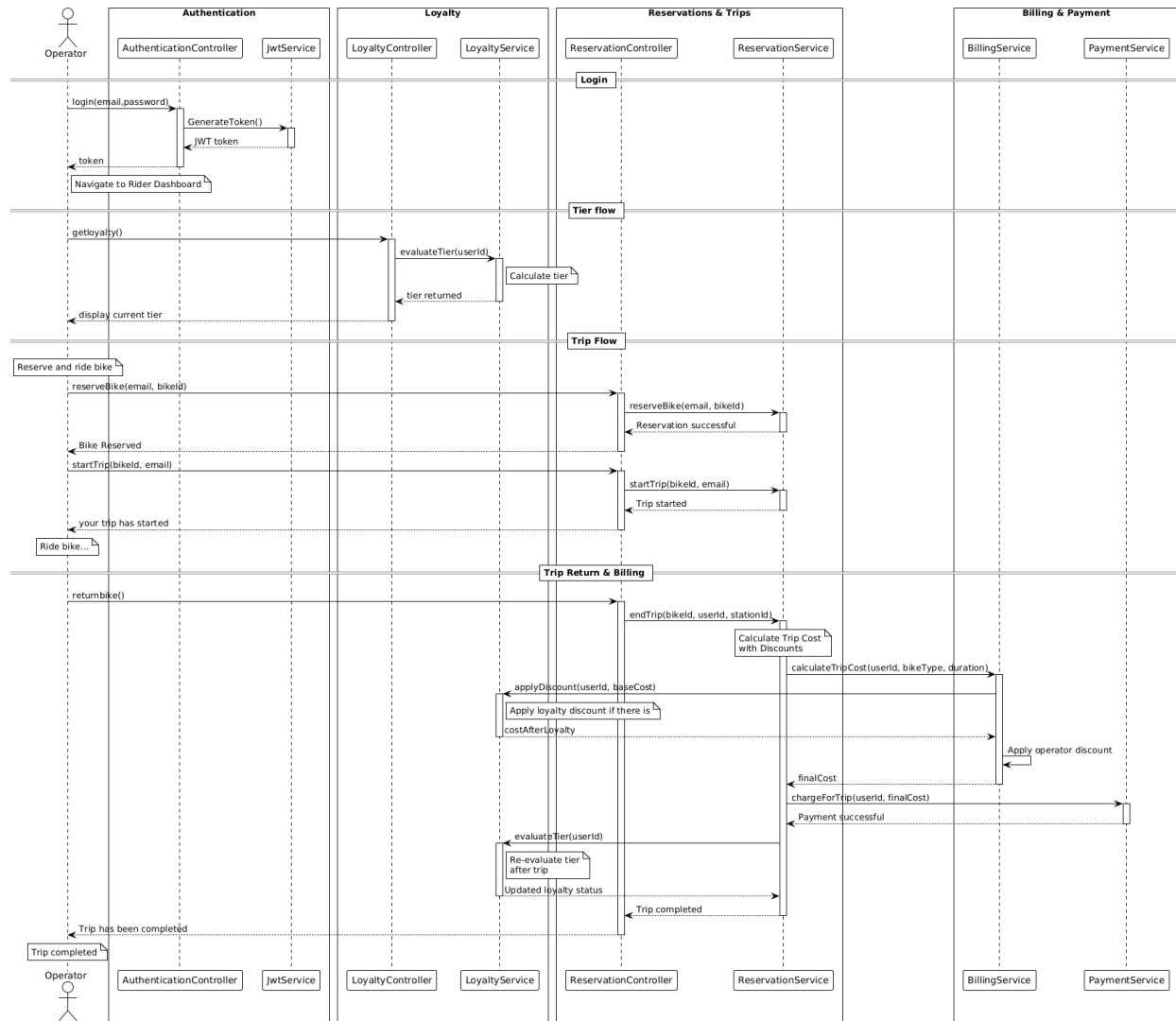
Returning bike at low occupancy station

Use Case Parameters	Details
Name	Return bike
Id	2
Actors	<u>Primary Actors:</u> Application User of type Rider <u>Secondary Actors:</u> Stripe
Description	When a rider returns a bike to a docking station that is less than 25% full, they would receive flex dollars that will serve as credit to the payment of the next ride. The trip would mark as completed and the rider would receive history and billing for that ride.
Preconditions	• The rider must have an account and not just be a one-time guest. • The rider must have a bike unlocked and is currently on a trip. • The destination docking station must have less than 25% capacity.
Stimulus/Trigger	The return bike button is clicked
Postconditions	• The ride is marked as completed • The rider is charged for the ride. • The rider receives history for this ride. • The rider receives a billing for this ride. • The rider has accumulated flex dollars for to serve as credit for their next ride. • The rider receives a confirmation that the ride has ended. • The rider receives notifications based on the loyalty tier and circumstances associated with them.
Basic Flow/Happy Path	• The rider click return bike. • The rider selects a destination docking station that is less than 25% of it's capacity • The rider confirms their selection.
Alternate Path	None
Exceptions	• If a rider is a one-time guest the do not accumulate flex dollars and they do not receive any notifications, tiers privileges, history or billing for their rides. They are given the option to become a member. • If a rider selects a docking station that is more than or is 25% full of its capacity, no flex dollars would be awarded for the user.

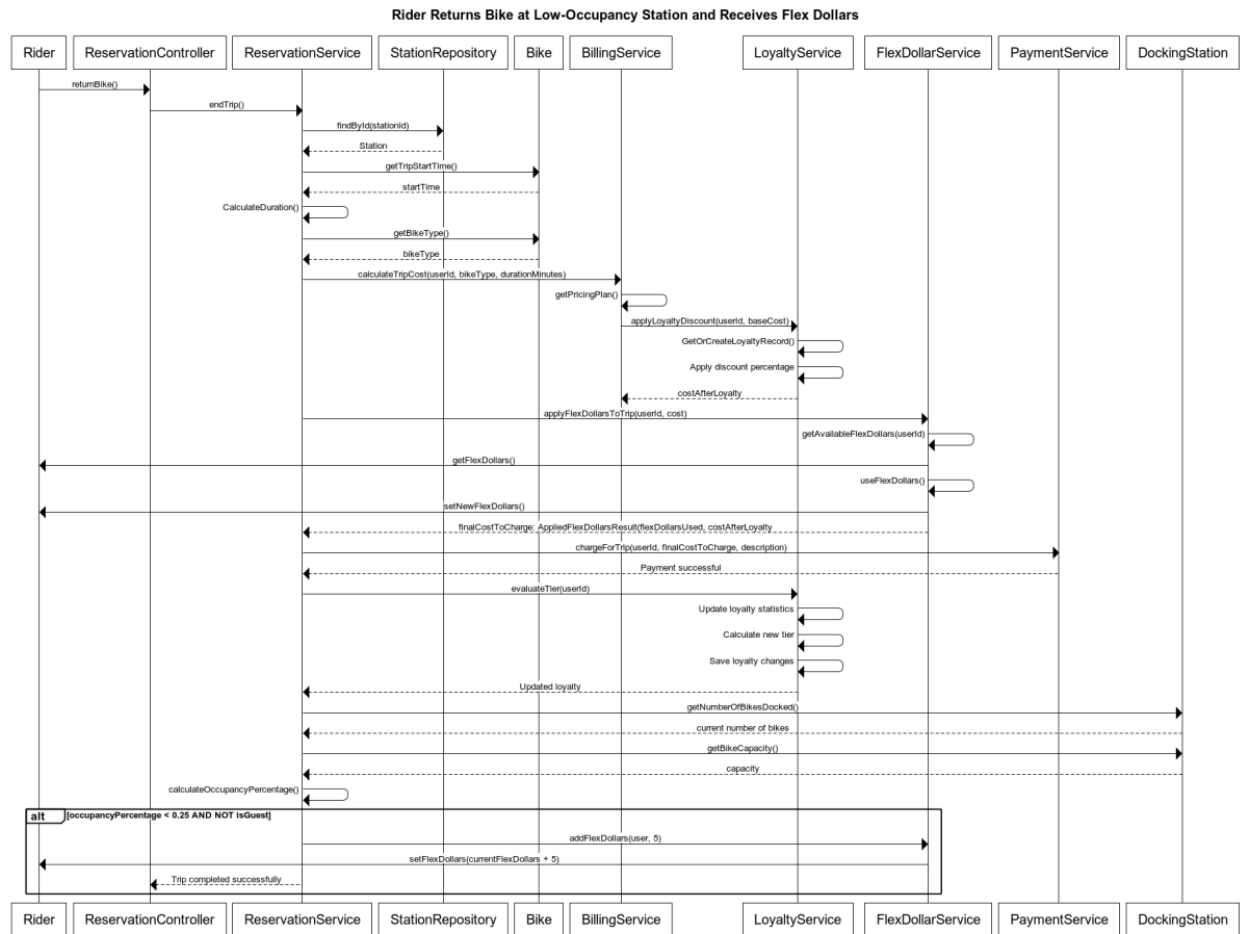
	• If a rider is not already on a ride, then the use case cannot initiate.
Minimal Guarantee	The ride is marked as completed, and the rider is charged.
Author	Youssef Yassa
Date	November 21 st , 2025

Sequence Diagrams

User Switching Views (Operator and Rider)



Returning bike at low occupancy station



Contribution Self-Declaration

All contributing members have contributed equally to this Phase delivery